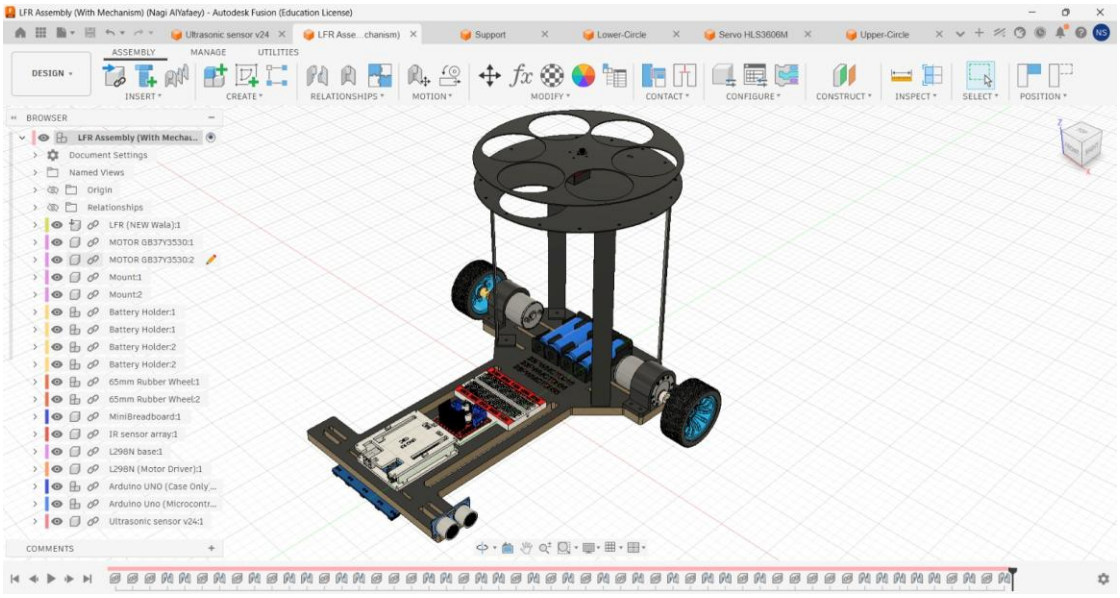
Line Following Robot with Ball-Putting Mechanism

Semester Project Report

Mechanical Design | Circuit Integration | Embedded Control



CAD assembly view of the LFR with ball-putting mechanism.

Project Type	Semester Project
Main Controller	Arduino Uno
Design Software	Autodesk Fusion 360 and Fritzing
Main Chassis	Laser-cut metal chassis, 35 x 30 cm
Team Members	Nagi Nasr, Huzaifa Taj, Abdlubasit

Table of Contents

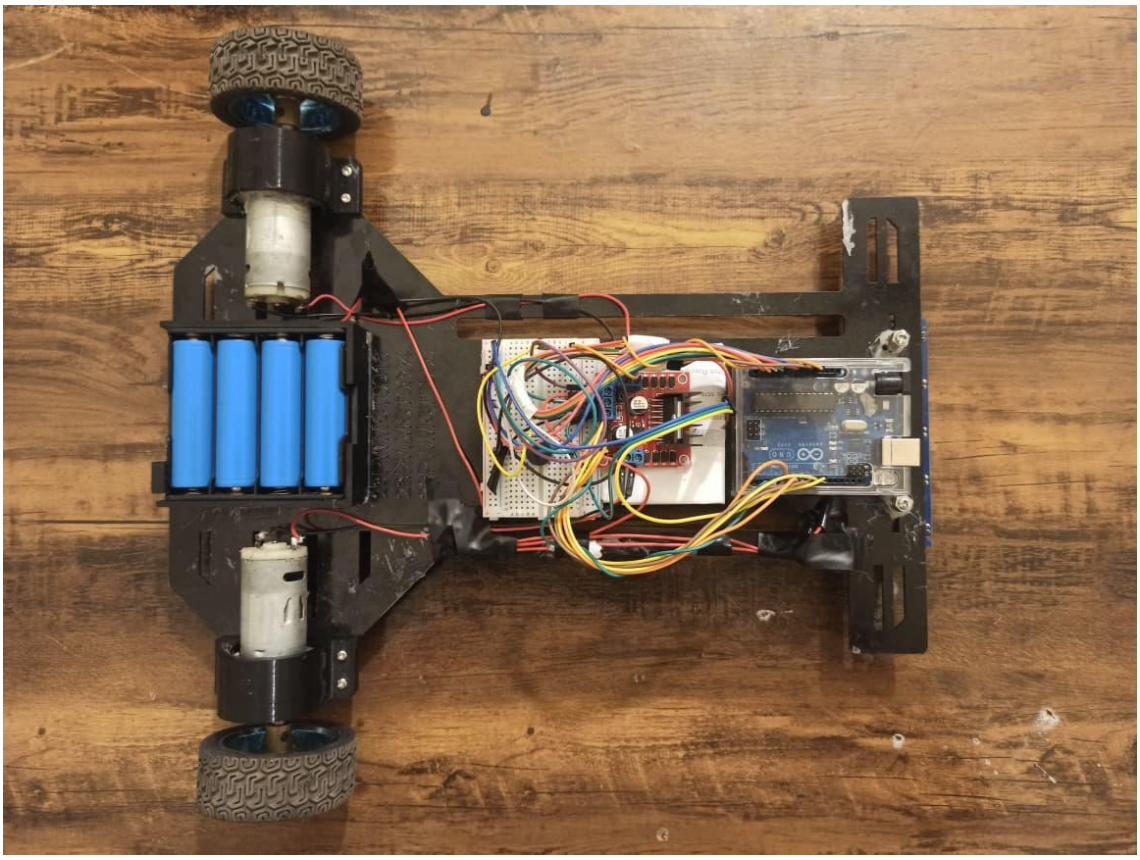
1. Abstract

This report presents the design and development of a line following robot with an added ball-putting mechanism. The project combines mechanical design, laser cutting, 3D printing, circuit design, sensor-based control, and Arduino programming. The robot follows a black line using a five-channel IR sensor array and controls two DC geared motors through an L298N motor driver. An ultrasonic sensor is used to detect boxes or target positions, and a servo motor is used to operate the ball-putting mechanism at selected angles. The mechanical assembly was designed in Autodesk Fusion 360, while the electronic wiring was planned using Fritzing. The main body was made from laser-cut metal with a size of 35 x 30 cm. Several parts were 3D printed, including the L298N base, four-cell battery holder, and DC motor mounts. The final system demonstrates a practical mechatronics project where design, fabrication, electronics, and embedded control work together in one complete platform.

2. Introduction

A line following robot is a basic but important mobile robot used to understand autonomous navigation. It follows a visible path on the ground by reading the contrast between the line and the surface. This project extends the normal LFR concept by adding a ball-putting mechanism. This means the robot is not only moving on a line; it also performs a simple task when it detects specific positions.

The project was developed as a semester project. The focus was not only on making the robot move, but also on building a complete engineering system. For this reason, the work included CAD design, metal chassis fabrication, 3D printed supports, circuit planning, wiring, code development, testing, and troubleshooting.



Final assembled robot top view.

3. Project Objectives

- Design a stable mobile robot chassis using Fusion 360.
- Fabricate the main chassis using laser-cut metal with 35 x 30 cm dimensions.
- Design and add 3D printed supports for the motor driver, battery holder, and DC motors.
- Use a five-channel IR sensor array for line detection.
- Control two DC geared motors using an L298N motor driver.
- Use ultrasonic sensing to detect boxes or target points.
- Operate a servo-based ball-putting mechanism when targets are detected.
- Prepare a clear circuit layout using Fritzing.
- Test the robot as a complete mechatronics system.

4. System Overview

The robot works by continuously reading the IR sensor array. The sensor values are used to calculate the position of the line. Based on this position, the Arduino controls the left and right motor speeds. When the ultrasonic sensor detects a box within the selected distance, the robot stops for a short time and moves the servo to the required angle. After this action, the robot continues line following.

4.1 Functional Block Diagram

IR Sensor Array	Arduino Uno	PID Logic	L298N Driver	DC Motors	Ultrasonic Sensor	Servo Mechanism

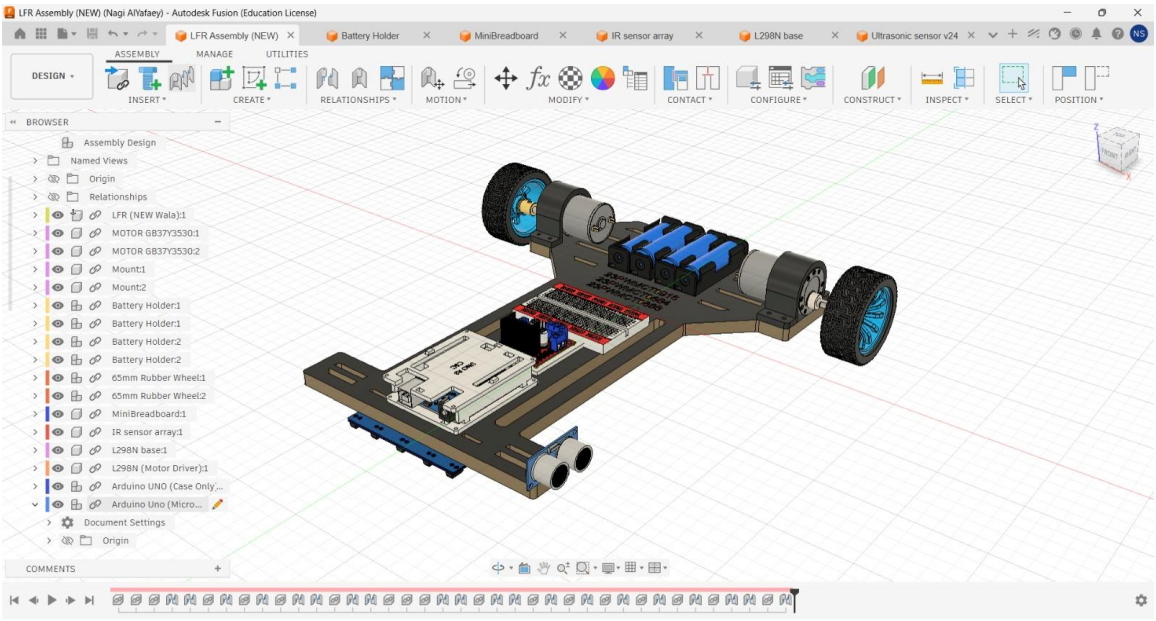
Main control flow: Sensors read the line and target distance. The Arduino processes the data and controls the motors and servo mechanism according to the program logic.

4.2 Major Subsystems

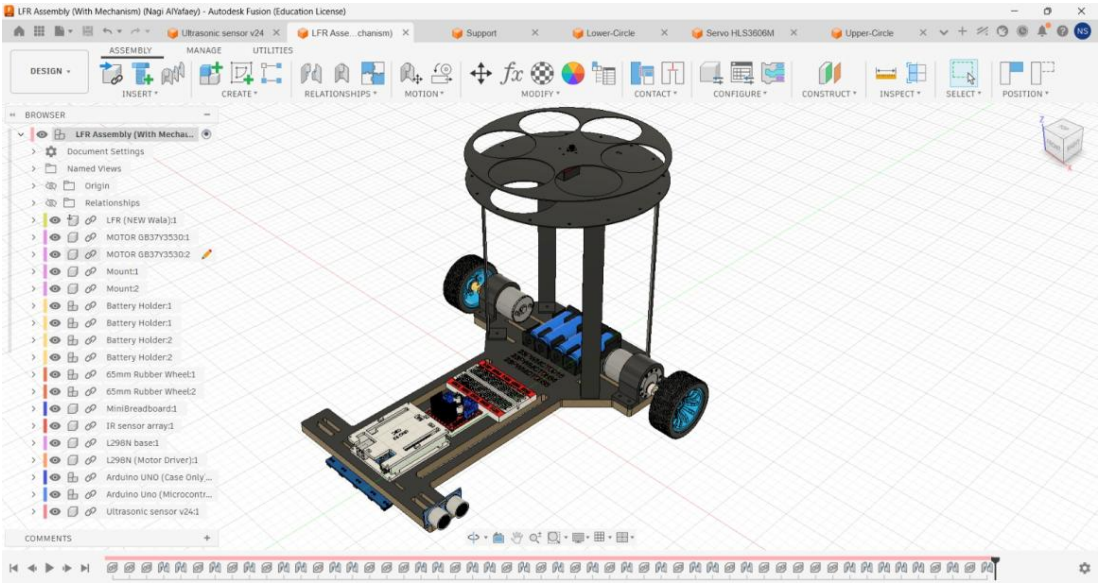
Subsystem	Main Parts	Purpose
Mechanical system	Laser-cut chassis, 3D printed mounts, wheels	Provides structure, alignment, and support.
Sensing system	IR sensor array, ultrasonic sensor	Detects line position and boxes/obstacles.
Control system	Arduino Uno and code logic	Processes sensor data and generates control signals.
Actuation system	DC motors, L298N, servo motor	Moves the robot and operates the ball-putting mechanism.
Power system	Four-cell battery holder	Supplies power to the robot system.

5. Mechanical Design and Fabrication

The mechanical design was prepared in Autodesk Fusion 360 before the physical assembly. This helped in placing the motors, battery holder, Arduino, motor driver, breadboard, sensors, and the ball-putting mechanism in a planned layout. The design process reduced fitting problems because each part position was checked in CAD before fabrication.



Fusion 360 main assembly without the ball-putting mechanism.



Fusion 360 assembly with the ball-putting mechanism.

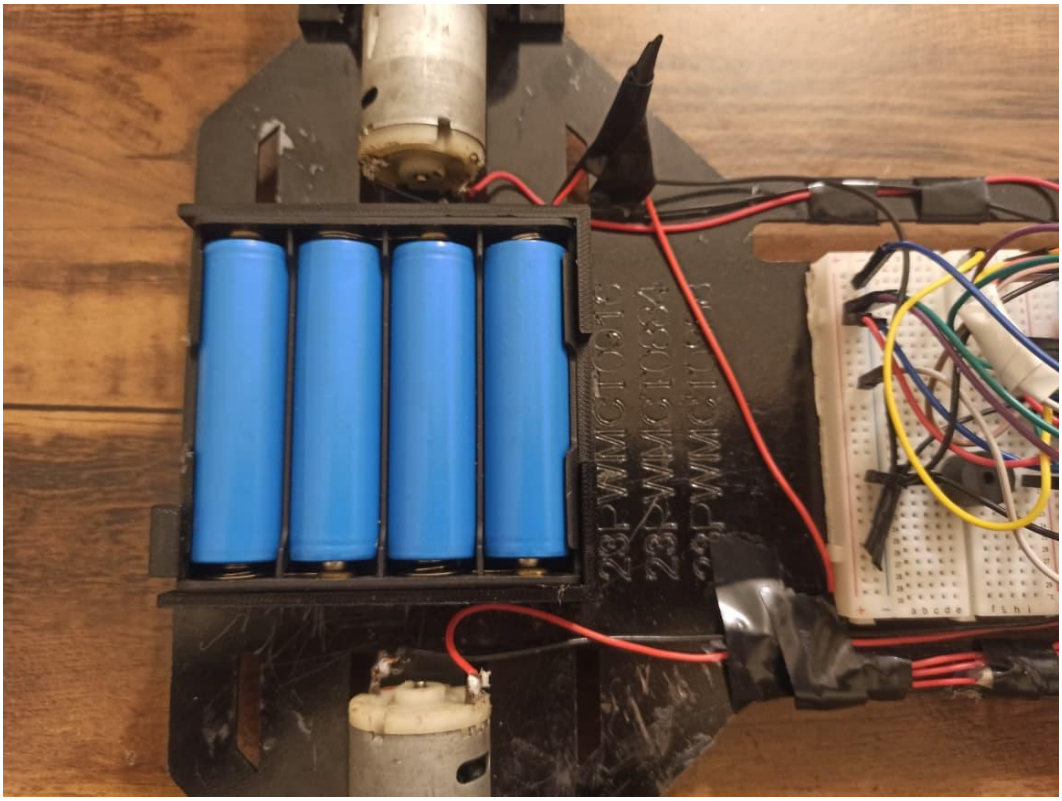
5.1 Main Chassis

The main case of the robot was made from laser-cut metal. The approximate chassis dimension is 35 x 30 cm. Metal was selected because it gives a strong base and reduces bending during movement. The chassis includes slots and mounting positions for the main parts.

5.2 3D Printed Parts

Some parts were 3D printed to improve assembly and to avoid direct contact between sensitive electronics and the metal body. The 3D printed parts included:

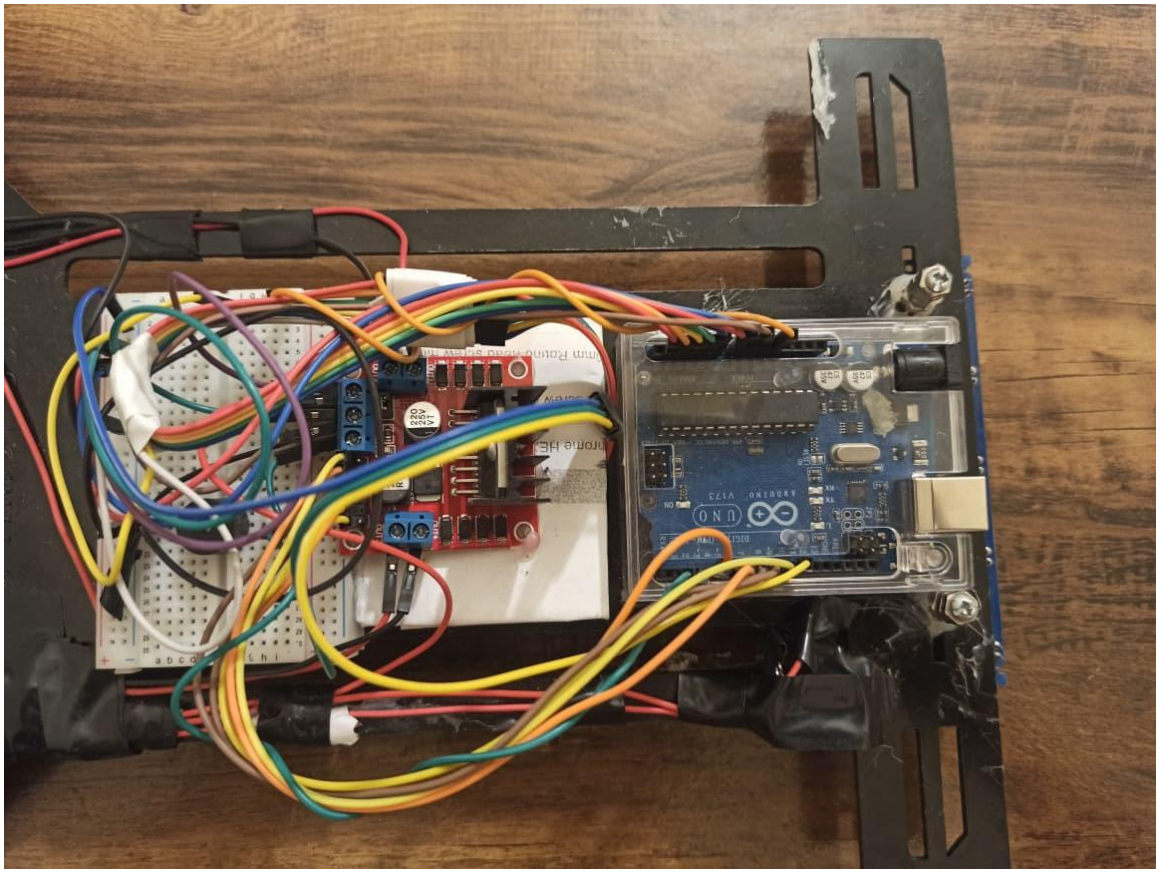
- L298N motor driver base, used to isolate the driver from the metal chassis.
- Four-cell battery holder for proper battery placement.
- DC motor mounts for holding the geared motors firmly.
- Mechanical supports for the ball-putting mechanism.



Four-cell battery holder mounted on the chassis.

6. Electronic Components

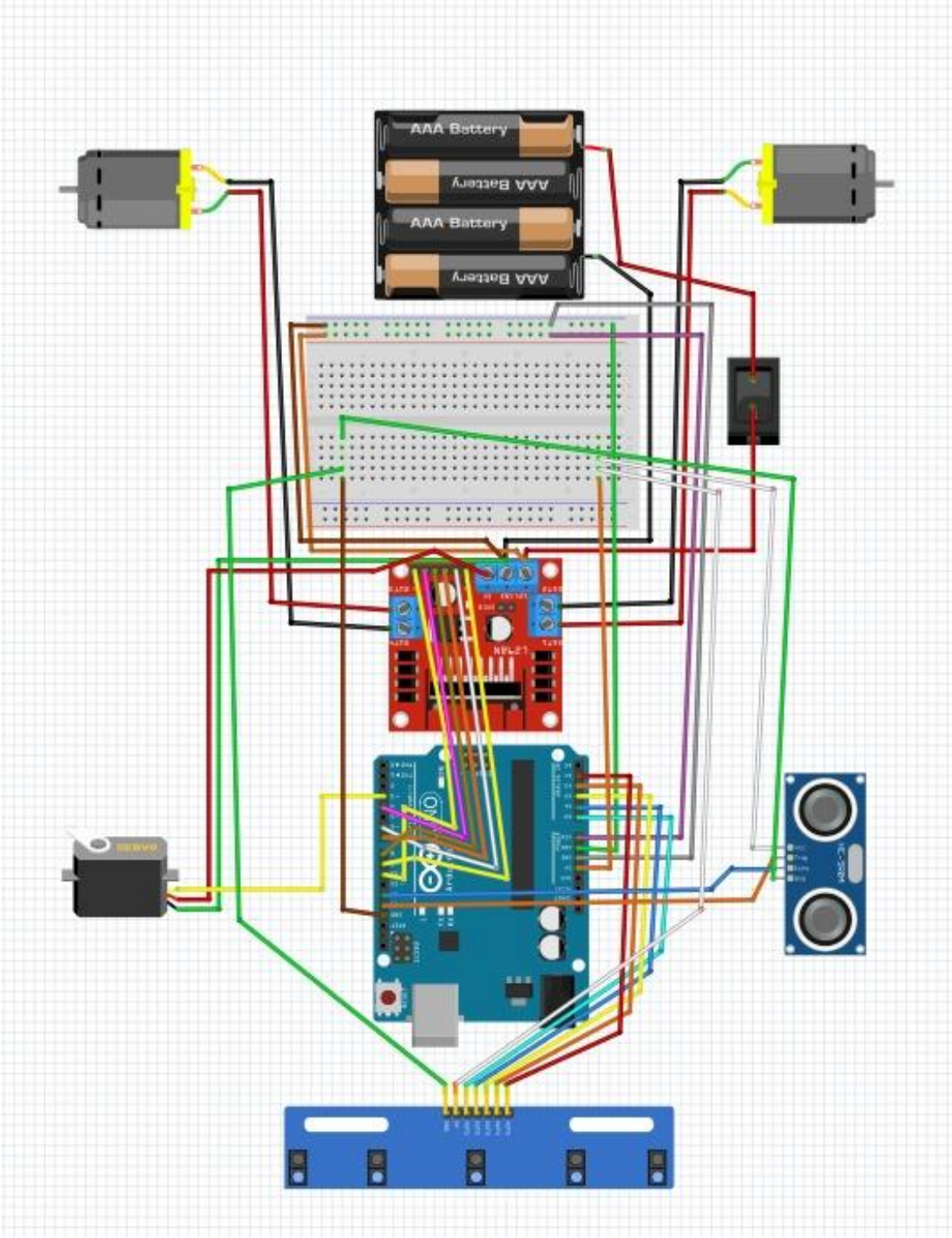
Component	Function
Arduino Uno	Main microcontroller used to read sensors and control outputs.
L298N motor driver	Controls direction and speed of the two DC geared motors.
DC geared motors	Provide movement for the robot wheels.
Five-channel IR sensor array	Detects the black line and gives the line position information.
Ultrasonic sensor	Detects boxes or target positions in front of the robot.
Servo motor	Moves the ball-putting mechanism to selected positions.
Buzzer	Provides an indication when a box is detected.
Battery holder	Holds the power cells used for the robot supply.
Breadboard and jumper wires	Used for signal distribution and quick circuit connection.



Main electronics area showing Arduino, L298N, breadboard, and wiring.

7. Circuit Design and Wiring

The circuit was planned using Fritzing before final assembly. The circuit includes the Arduino Uno, L298N motor driver, DC motors, IR sensor array, ultrasonic sensor, servo motor, buzzer, switch, and battery supply. Planning the circuit in Fritzing made it easier to understand the connections and reduce wiring mistakes during physical implementation.



Fritzing circuit diagram used for wiring planning.

7.1 Pin Configuration Used in Code

Signal	Pin(s)	Purpose
IR1 to IR5	A0, A1, A2, A3, A4	Line sensor inputs
ENA, ENB	9, 10	PWM speed control for left and right motors
IN1, IN2	6, 7	Motor direction control for one side
IN3, IN4	8, 4	Motor direction control for other side

Ultrasonic TRIG, ECHO	13, 12	Distance measurement
Servo	3	Ball-putting mechanism control
Buzzer	2	Box detection indication

8. Control Algorithm and Code Explanation

The robot uses a PID-based line following method. The IR sensor array gives five digital readings. In the code, a black line is treated as an active reading. Each sensor is given a weight so the program can estimate whether the line is on the left, center, or right side of the robot. The calculated error is then used to adjust motor speed.

8.1 PID Logic

The proportional term reacts to the present error. The integral term is set to zero in this project, so it does not strongly affect the control. The derivative term reacts to change in error and helps reduce sudden oscillation. The code also uses dynamic base speed, which decreases speed during curves and keeps better control while turning.

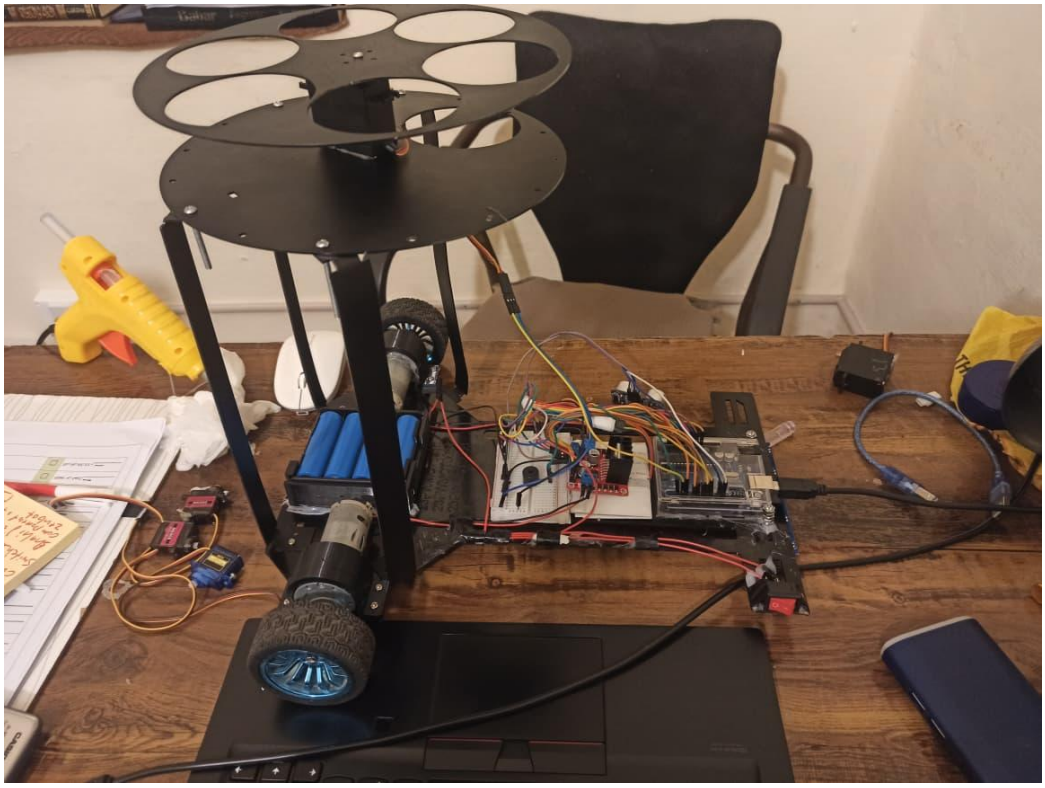
Parameter	Value / Description
Kp	14.0
Ki	0.0
Kd	20.0
Straight speed	210
Base speed	225
Maximum speed	255
Minimum curve speed	160
Pivot speed	230

8.2 Line Loss Recovery

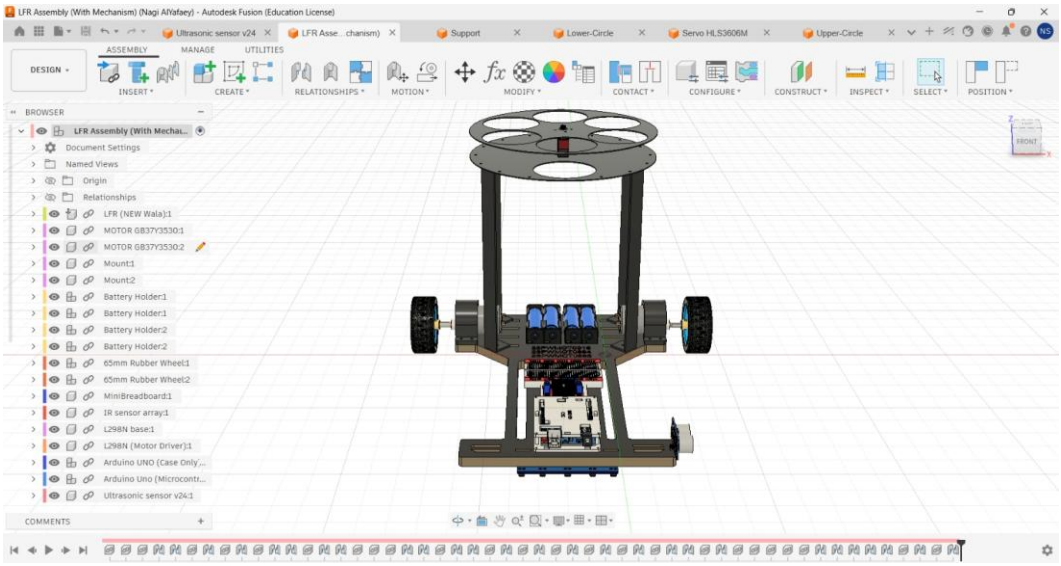
If the robot loses the line, the code checks the last known error. If the last error was on the left side, the robot pivots left. If it was on the right side, the robot pivots right. This helps the robot search for the line again instead of stopping completely.

9. Ball-Putting Mechanism

The ball-putting mechanism is controlled using a servo motor. In the code, the servo starts from 0 degree when the system turns on. When the ultrasonic sensor detects a box or target within the selected distance, the robot stops shortly and moves the servo to a specific angle. The program is prepared for three box positions: 65 degrees for the first box, 130 degrees for the second box, and 180 degrees for the third box.



Physical assembly showing the ball-putting mechanism mounted on the robot.



CAD front view of the ball-putting mechanism.

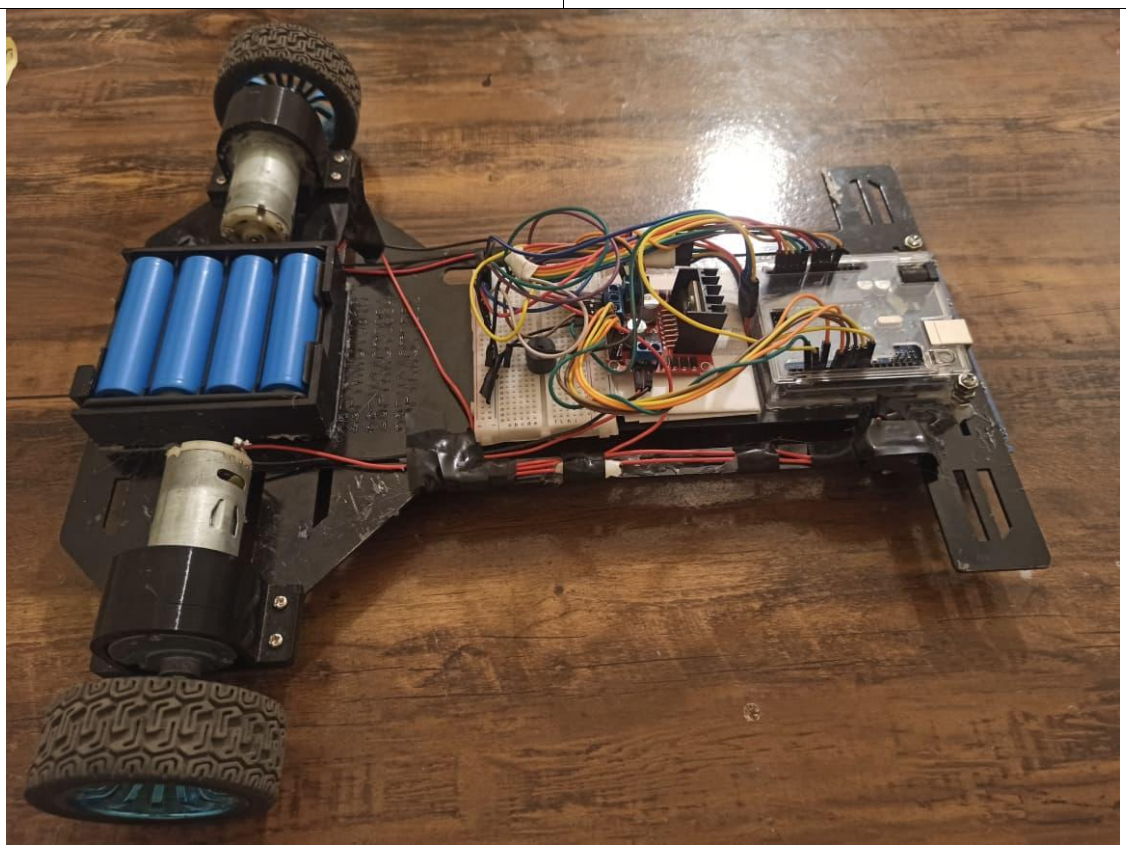
9.1 Box Detection Sequence

1. Robot follows the line normally.
2. Ultrasonic sensor measures distance continuously.
3. If a box is detected within 20 cm, the robot stops briefly.
4. Servo moves to the required angle according to the box count.
5. Buzzer turns ON as an indication.
6. Robot resumes line following.

10. Testing and Results

The robot was tested after integrating the mechanical body, wiring, and code. The main tests were line detection, motor direction, motor speed, turning response, ultrasonic detection, servo movement, and complete system behavior. The final prototype was able to combine line following with target detection and servo-based action.

Test Area	Observed Result
Line detection	IR sensor array detected the line and provided digital values.
Motor control	L298N driver controlled motor direction and PWM speed.
Turning behavior	PID correction helped the robot adjust on curves.
Line loss recovery	Pivot logic helped the robot search for the line again.
Box detection	Ultrasonic sensor detected close objects within the selected range.
Servo action	Servo moved to different angles for the ball-putting sequence.
Mechanical stability	Laser-cut chassis and motor mounts provided a stable structure.



Side view of the completed robot during assembly/testing.

11. Problems Faced and Improvements

During the project, several practical issues were expected because the system contains mechanical, electrical, and programming parts together.

Issue	Improvement

Wiring complexity	Use shorter wires, cable ties, and clear color coding.
Metal chassis and electronics contact	Keep using 3D printed isolation bases for drivers and electronic boards.
Sensor calibration	Test the IR sensor height and threshold on the real track before final run.
Motor speed difference	Tune motor speeds and PID values after observing actual movement.
Battery placement	Keep the battery holder balanced to reduce unwanted tilt.
Servo movement timing	Adjust servo delay and target angles according to the real ball-putting mechanism.

11.1 Future Improvements

- Replace temporary breadboard wiring with a soldered PCB or perfboard.
- Use better cable management for cleaner assembly and lower risk of loose connections.
- Add adjustable sensor height for easier calibration.
- Use a more efficient motor driver if higher current motors are required.
- Improve the ball-putting mechanism with more accurate mechanical indexing.
- Add an LCD or small OLED display for live status such as box count and distance.

12. Conclusion

The Line Following Robot with Ball-Putting Mechanism was successfully designed and developed as a semester project. The project demonstrates a complete practical mechatronics workflow: CAD design, laser-cut fabrication, 3D printed parts, circuit planning, Arduino programming, sensor feedback, motor control, and servo-based task execution. The use of Fusion 360 helped in accurate component placement, while Fritzing helped in understanding and planning the circuit. The final robot is a strong learning platform for robotics, embedded systems, and mechanical design.

Special thanks to Huzaifa Taj and Abdlubasit for their contribution and teamwork during the completion of this project.

Appendix A: Arduino Code

The following code was used for the line following robot, ultrasonic box detection, buzzer indication, and servo-based ball-putting action.

```
//By: Nagi Al-Yafaey

#include <Arduino.h>

// --- Pin Definitions ---
#define IR1 A0
#define IR2 A1
#define IR3 A2
#define IR4 A3
#define IR5 A4

#define ENA  9
#define IN1  6
#define IN2  7
#define ENB 10
#define IN3  8
#define IN4  4

// --- Ultrasonic Pins ---
#define TRIG_PIN 13
#define ECHO_PIN 12

// --- One Servo Pin ---
#define SERVO_PIN 3

// --- Buzzer Pin ---
#define BUZZER_PIN 2

// --- PID Constants ---
float Kp = 14.0;
float Ki = 0.0;
float Kd = 20.0;

// --- Speed Settings ---
int straightSpeed = 210;
int baseSpeed     = 225;
int maxSpeed      = 255;
int minSpeed      = -190;
int pivotSpeed    = 230;

// --- Dynamic Speed Settings ---
int minCurveSpeed = 160;

// --- Straight Line Stability ---
float straightDeadband = 0.45;

// --- Global Variables ---
float prevError = 0;
float lastError = 0;
float integral = 0;
float filteredDerivative = 0;

bool debugMode = false;

// --- Ultrasonic and Box Settings ---
int boxCount = 0;
int obstacleConfirmCount = 0;

unsigned long lastBoxTime = 0;
unsigned long lastPrintTime = 0;
unsigned long lastDistanceReadTime = 0;
```

```
long currentDistance = -1;

const int OBSTACLE_DISTANCE = 20;
const int MAX_BOXES = 3;
const int COOLDOWN_TIME = 4000;
const int REQUIRED_CONFIRM = 1;
const int PRINT_INTERVAL = 400;
const int DISTANCE_INTERVAL = 25;

// --- Servo Angle Settings ---
int currentAngle = 0;

const int ANGLE_START = 0;
const int ANGLE_BOX_1 = 65;
const int ANGLE_BOX_2 = 130;
const int ANGLE_BOX_3 = 180;

// Wider pulse range, closer to Arduino Servo library behavior
const int SERVO_MIN_PULSE = 544;     // approx 0 degree
const int SERVO_MAX_PULSE = 2400;    // approx 180 degree

// --- Function Prototypes ---
void driveMotors(int leftSpeed, int rightSpeed);
void pivotRight();
void pivotLeft();
void runPID(int s1, int s2, int s3, int s4, int s5);
bool lineDetected(int s1, int s2, int s3, int s4, int s5);

void updateDistance();
long getDistance();

void handleBox(int boxNumber);
void moveServoToAngle(int targetAngle);
void holdServoAngle(int angle, int holdTimeMs);
int angleToPulse(int angle);

void playBuzzer();
void printStatus(long distance);

void setup() {
  Serial.begin(9600);

  // Higher PWM frequency for smoother motor response on D9 and D10
  TCCR1B = TCCR1B & 0b11111000 | 0x01;

  pinMode(IR1, INPUT);
  pinMode(IR2, INPUT);
  pinMode(IR3, INPUT);
  pinMode(IR4, INPUT);
  pinMode(IR5, INPUT);

  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  digitalWrite(TRIG_PIN, LOW);

  pinMode(SERVO_PIN, OUTPUT);
  digitalWrite(SERVO_PIN, LOW);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
```

```
  pinMode(LED_BUILTIN, OUTPUT);

  // IMPORTANT: Set servo to 0 degree at startup
  currentAngle = ANGLE_START;
  holdServoAngle(currentAngle, 1500);
  digitalWrite(SERVO_PIN, LOW);

  for (int i = 0; i < 3; i++) {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(80);
    digitalWrite(LED_BUILTIN, LOW);
    delay(80);
  }

  Serial.println("================================");
  Serial.println(" Complete LFR + Ball Potting System");
  Serial.println(" Servo starts at 0 degree");
  Serial.println(" Box 1 = 65 | Box 2 = 130 | Box 3 = 180");
  Serial.println(" Ultrasonic Distance: 15 cm");
  Serial.println("================================");

  delay(500);
}

void loop() {
  int s1 = digitalRead(IR1);
  int s2 = digitalRead(IR2);
  int s3 = digitalRead(IR3);
  int s4 = digitalRead(IR4);
  int s5 = digitalRead(IR5);

  updateDistance();

  if (debugMode && millis() - lastPrintTime >= PRINT_INTERVAL) {
    printStatus(currentDistance);
    lastPrintTime = millis();
  }

  if (currentDistance > 0 && currentDistance <= OBSTACLE_DISTANCE) {
    obstacleConfirmCount++;
  } else {
    obstacleConfirmCount = 0;
  }

  if (obstacleConfirmCount >= REQUIRED_CONFIRM &&
      boxCount < MAX_BOXES &&
      millis() - lastBoxTime > COOLDOWN_TIME) {

    boxCount++;
    handleBox(boxCount);

    lastBoxTime = millis();
    obstacleConfirmCount = 0;
  }

  if (!lineDetected(s1, s2, s3, s4, s5)) {
    integral = 0;
    filteredDerivative = 0;

    if (lastError < 0) {
      pivotLeft();
    } else {
      pivotRight();
    }
  } else {
    runPID(s1, s2, s3, s4, s5);
  }
}
```

```
bool lineDetected(int s1, int s2, int s3, int s4, int s5) {
  return (s1 == 0 || s2 == 0 || s3 == 0 || s4 == 0 || s5 == 0);
}

void runPID(int s1, int s2, int s3, int s4, int s5) {
  int v1 = 1 - s1;
  int v2 = 1 - s2;
  int v3 = 1 - s3;
  int v4 = 1 - s4;
  int v5 = 1 - s5;

  float weightedSum = (-5 * v1) + (-2 * v2) + (0 * v3) + (2 * v4) + (5 * v5);
  int sensorsOnLine = v1 + v2 + v3 + v4 + v5;

  float error = 0;

  if (sensorsOnLine > 0) {
    error = weightedSum / sensorsOnLine;
    lastError = error;
  }

  if (abs(error) <= straightDeadband) {
    integral = 0;
    filteredDerivative = 0;
    prevError = error;

    driveMotors(straightSpeed, straightSpeed);
    return;
  }

  integral += error;
  integral = constrain(integral, -20, 20);

  float rawDerivative = error - prevError;
  filteredDerivative = (0.85 * filteredDerivative) + (0.15 * rawDerivative);

  float correction = (Kp * error) + (Ki * integral) + (Kd * filteredDerivative);

  prevError = error;

  int dynamicBaseSpeed = baseSpeed - (abs(error) * 7);
  dynamicBaseSpeed = constrain(dynamicBaseSpeed, minCurveSpeed, baseSpeed);

  int leftSpeed  = dynamicBaseSpeed + (int)correction;
  int rightSpeed = dynamicBaseSpeed - (int)correction;

  leftSpeed  = constrain(leftSpeed, minSpeed, maxSpeed);
  rightSpeed = constrain(rightSpeed, minSpeed, maxSpeed);

  driveMotors(leftSpeed, rightSpeed);
}

void driveMotors(int left, int right) {
  if (left >= 0) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
  } else {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    left = -left;
  }

  if (right >= 0) {
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
  } else {
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
```

```
    right = -right;
  }

  left = constrain(left, 0, 255);
  right = constrain(right, 0, 255);

  analogWrite(ENA, left);
  analogWrite(ENB, right);
}

void pivotRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);

  analogWrite(ENA, pivotSpeed);
  analogWrite(ENB, pivotSpeed);
}

void pivotLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);

  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  analogWrite(ENA, pivotSpeed);
  analogWrite(ENB, pivotSpeed);
}

void updateDistance() {
  if (millis() - lastDistanceReadTime >= DISTANCE_INTERVAL) {
    currentDistance = getDistance();
    lastDistanceReadTime = millis();
  }
}

long getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 12000);
  // 12000 us is enough for around 200 cm, faster than 25000

  if (duration == 0) {
    return -1;
  }

  long distance = duration * 0.0343 / 2;

  if (distance < 2 || distance > 200) {
    return -1;
  }

  return distance;
}

void handleBox(int boxNumber) {
  Serial.println();
  Serial.print("*** BOX DETECTED: ");
  Serial.print(boxNumber);
  Serial.println(" ***");
```

```
  driveMotors(0, 0);
  delay(450);

  if (boxNumber == 1) {
    moveServoToAngle(ANGLE_BOX_1);
  }
  else if (boxNumber == 2) {
    moveServoToAngle(ANGLE_BOX_2);
  }
  else if (boxNumber == 3) {
    moveServoToAngle(ANGLE_BOX_3);
  }
  else if (boxNumber == 4) {
    Serial.println("Fourth box detected. No servo movement.");
  }

  playBuzzer();

  Serial.println("*** RESUMING LINE FOLLOWING ***");
  Serial.println();

  delay(350);
}

void moveServoToAngle(int targetAngle) {
  targetAngle = constrain(targetAngle, 0, 180);

  int stepValue = 1;

  if (currentAngle < targetAngle) {
    for (int angle = currentAngle; angle <= targetAngle; angle += stepValue) {
      holdServoAngle(angle, 10);
    }
  } else {
    for (int angle = currentAngle; angle >= targetAngle; angle -= stepValue) {
      holdServoAngle(angle, 10);
    }
  }

  currentAngle = targetAngle;

  // Hold final angle shortly, then stop signal
  holdServoAngle(currentAngle, 500);
  digitalWrite(SERVO_PIN, LOW);

  Serial.print("Servo moved to angle: ");
  Serial.println(currentAngle);
}

void holdServoAngle(int angle, int holdTimeMs) {
  int pulseWidth = angleToPulse(angle);
  unsigned long startTime = millis();

  while (millis() - startTime < holdTimeMs) {
    digitalWrite(SERVO_PIN, HIGH);
    delayMicroseconds(pulseWidth);

    digitalWrite(SERVO_PIN, LOW);
    delayMicroseconds(20000 - pulseWidth);
  }

  digitalWrite(SERVO_PIN, LOW);
}

int angleToPulse(int angle) {
  angle = constrain(angle, 0, 180);
  return map(angle, 0, 180, SERVO_MIN_PULSE, SERVO_MAX_PULSE);
}
```

```
void playBuzzer() {
  digitalWrite(BUZZER_PIN, HIGH);
  delay(1200);
  digitalWrite(BUZZER_PIN, LOW);
}

void printStatus(long distance) {
  Serial.print("Distance: ");

  if (distance > 0) {
    Serial.print(distance);
    Serial.print(" cm");
  } else {
    Serial.print("No echo");
  }

  Serial.print(" | Boxes: ");
  Serial.print(boxCount);
  Serial.print("/4");

  Serial.print(" | Servo Angle: ");
  Serial.println(currentAngle);
}
```